

Tensor Networks For Multi-Agent Reinforcement Learning



Author: Mustafa Omar

Supervisor: Dr Edward Gillman

MSc: Machine Learning in Science

The School of Physics and Astronomy

The University of Nottingham

July 26, 2026

Abstract

In this work we investigate the use of Tensor Networks for the problem of Multi-Agent reinforcement learning, where we utilise a non-shared policy for a set of 20 agents. We train our baseline model on the Piston Ball environment from Petting zoo, and find that the model manages to successfully learn the environment, obtaining an average scores of 1965.3 ± 8.6 , 1092 ± 419.0 and -72.2 ± 262.2 for the deterministic, non-deterministic and random policies respectively. We construct a novel Tensor Network architecture, and successfully carry out a forward and backwards pass, however, we fail at successfully learning the environment using the PPO algorithm. Furthermore during experimentation we find that the Tensor Network linearly scales with the number of agents for inference time and number of parameters, but, it exponentially scales with the Intra-Bond/Contraction dimension size for the inference time and number of parameters.

Contents

1	Introduction	5
1.1	Multi-Agent Reinforcement Learning	5
1.2	Related Work	6
1.3	Contribution	7
1.4	Github	7
2	Markov Decision Process	9
2.1	Environment: Trajectories And Properties	9
2.2	Markov Decision Process And Decision Policies	10
3	Actor Critic Methods	13
3.1	Actor Critic	13
3.2	Policy Gradients	14
4	Defining Tensor Networks	17
4.1	Definition Of A Tensor	17
4.2	Definition Of A Tensor Network	18
4.3	Applied Tensor Networks	21
5	Problem Setting And Baseline Solution	23

5.1	Petting Zoo	23
5.2	Pistonball	23
5.3	Environment Reward Mechanics	24
5.4	Preprocessing	25
5.5	Baseline Solution	25
5.6	PPO configurations	28
5.7	Quantitative Measure	28
6	An Implementation Of Tensor Networks For MARL	29
6.1	Why Jax ?	29
6.2	Complete Architecture And Visualisation	30
6.3	The Value Head: State value	31
6.4	The Policy Head	32
6.5	Executing A Forward & Backwards Pass	32
6.6	Modifying PPO	33
7	Discussion & Results	36
7.1	Baseline Training Summary & Tensor Network Learning Hypothesis	36
7.1.1	Baseline Training Summary	36
7.1.2	Tensor Network Learning Hypothesis	37
7.2	Scaling	38
7.2.1	Number Of Agents	38
7.2.2	Intra-Bond Dimension Size	40
7.2.3	Image Size	41
7.3	Learning	42
8	Conclusion	43

Chapter 1

Introduction

1.1 Multi-Agent Reinforcement Learning

Reinforcement learning is a class of machine learning methods, its primary concern is with teaching an agent how to interact with an environment, in order to maximise an environment reward function. This objective is achieved by crafting a policy π^* that the agent uses to traverse the states of the environment in the most rewarding way possible (figure 1.1).

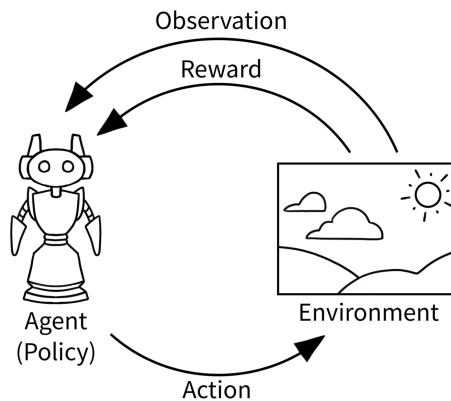


Figure 1.1: sourced from [1], showing how a single agent policy interacts with the environment; returning an action after receiving the observation and reward.

Furthermore, a generalisation of Reinforcement learning is the problem of Multi-Agent

Reinforcement Learning (MARL) (figure 1.2), where instead of having a single agent within the environment, we have many actors, each of whom observes their own part of the environment and in some cases have their own unique policy.

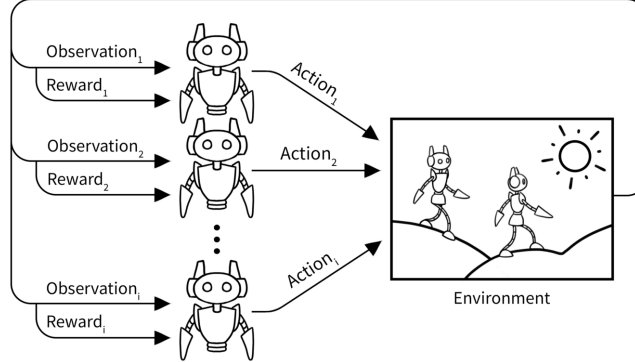


Figure 1.2: sourced from [1], showing how a multi-agent non-shared policy interacts with the environment; outputting an action per agent after each agent receives their own observation and reward, with computations occurring simultaneously.

In this investigation we look at the application of Tensor Networks to the problem of multi-agent reinforcement learning. The Tensor Network operates in such a way that allows parameter sharing between agents, therefore, in the context of multi-agent reinforcement learning, we hypothesise that this parameter sharing would allow for communication between the agents, and thus provide a greater learning efficiency. The goal of this investigation is to construct the Tensor Network and setup a baseline to which it can be compared.

1.2 Related Work

Tensor Networks are sparse data structures engineered for the efficient representation and manipulation of very high-dimensional data [2], making them advantageous for use in quantum physics [3, 4, 5], statistical mechanics [6, 7], quantum chemistry [8, 9], quantum field theory and gravity [10, 11]; and cosmology [12, 13, 14]. Their ability to represent high-dimensional data is thought to also make them an ideal candidate for use in machine learning, applications saw the use of Tensor Networks in classification on the MNIST

dataset [15], a theoretical comparison with convolutional neural networks [16] as well as a body of Machine Learning related applications (see [2]).

Tensor Networks also saw an implementation as a general library for physics and machine learning [2], the purpose of this was to accelerate research by creating an easy framework, on top of which Tensor Network systems could easily be implemented. Furthermore, for reinforcement learning, a couple of works that are specifically relevant to our investigation include: Y. F. and Xiao-Yang Liu’s utilisation of Tensor Networks for variational reinforcement learning [17], and the formal definition relating Tensor Networks to Finite Markov Decision Process (FMDP) by Gillman, Edward Et al. [18], this lays the foundation of our investigation and we extend on this work by applying Tensor Networks to the problem of Multi Agent Reinforcement learning under a Finite Markov Decision Process.

1.3 Contribution

In this work, we contribute by constructing and using the following for the purpose of Multi-Agent reinforcement learning using Jax [19] and the DeepMind Ecosystem [20]: CNNs for (per agent) feature extraction, Feed-forward (MLP) layers on top of the feature extractor for agent wise inference, Tensor Networks on top of the feature extractor for discrete agent wise inference, this includes constructing the value network head and the policy network head in Jax. Furthermore, we apply StableBaselines3 [21, 1] and ACME [22] for an Actor-Critic Multi-Agent reinforcement learning baseline, using the PPO algorithm [23] and the aforementioned CNN architecture. This is followed by adaptations to the PPO algorithm specifically for Tensor Networks.

1.4 Github

We release the code on our Github page [24], here we include all of the scripts used to train the different models as well as scripts used to evaluate the models after training, furthermore, we also include the Jax implementation of the PPO algorithm (Proximal

Policy optimisation), as well as the trainer and other minor utility required to construct, run and evaluate the reinforcement learning agent. To successfully and reproducibly run the code on any machine, we wrap the source code around docker, using docker will automatically install the dependencies and any other applications required to run the source code, furthermore, a GPU Dockerfile and docker-compose files are included, such that training on a GPU is also made easy, this is because there are frequent problems with aligning Cuda, Cudnn, Jax and Jaxlib versions to work with the GPU, docker resolves all of the overhead associated with that, allowing us to easily use different machines both locally and in the cloud in order to train and test our models.

Chapter 2

Markov Decision Process

2.1 Environment: Trajectories And Properties

A major part of Reinforcement learning is the environment, the environment provides the set of actions that the agent can take at any moment in time $A_t \in A$ this is a subset of all the available actions A within the environment, similarly, following an action selection a_{t-1} by the agent in the the state s_{t-1} , the environment then provides the reward $r_t(s_{t-1}|a_{t-1}) \in R$ where the reward is an environment specific function (see section 5.3); as well as the next state $s_t(s_{t-1}|a_{t-1}) \in S$, where the new state and the reward is conditionally (and is only) dependant on the state and action chosen at time $T = t - 1$ (Note: This property is due to the assumption of a Markov Decision processes see section 2.2). This process repeats itself as the agent selects actions and is known as a trajectory, it could be generalised as follows: At time $T = 0$ the agent is provided an initial state s_0 , for which it picks an action a_0 , and the trajectory progresses as follows:

$$\omega_0^T = s_0, a_0 \rightarrow s_1, a_1, r_1 \rightarrow s_2, a_2, r_2 \rightarrow s_2, a_2, r_2 \rightarrow \dots \quad (2.1)$$

and the rewards could be summed such that:

$$G_{1 \rightarrow T} = \sum_{t=1}^T r_t \quad (2.2)$$

where G is the return for the trajectory starting at (s_0, a_0) , t is the current time step and T the index of the final time step. This metric is used to obtain the agents performance

for a single trajectory, adding to this, we could obtain the agents overall performance by averaging the return for many trajectories, this is known as the expected return and is defined as:

$$\mathbb{E} [G_{1 \rightarrow T}^I] = \frac{\sum_{i=1}^N G_{1 \rightarrow T}^i}{N} \quad (2.3)$$

Where $\mathbb{E}$ is the expectation and N is the total number of trajectories.

Furthermore, we can now formally introduce the state value function. The state value function is a quantitative measure of how good the current state is, we obtain this measure by calculating the expected return under a policy π given the state s ; the state value function is defined with respect to the state and how the agents behaves, and the agent's behaviour is governed by a behaviour/action generating function known as the policy π (see section 2.2 for definition), we therefore define the state value function as follows:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right] \quad (2.4)$$

where v_{π} is the state value function for the policy π , t is the time step at which the state is provided and the action is generated, and γ is the discount constant which results in a discounted reward sum, this makes the agent value immediate rewards more than rewards it receives later. We primarily utilise this state value function, however, for more specific purposes the state-action value function could be used; a value function conditioned on the state and the action may be used, see [25] for details on the state-action value function.

We have mentioned that the agent is capable of selecting actions, in section 2.2 we formally introduce Markov decision processes and decision policies, this should clarify how the environment transition probabilities are structured, as well as the means by which the agent selects actions.

2.2 Markov Decision Process And Decision Policies

The environment is formally described as a Markov decision process (MDP) [26], where if given a state s and an action a it will provide a stochastic transition; a transition probability distribution, that describes the probability of the next state $s' \in S$ that the agent will be transitioning to given this action, with each action providing a different

probability distribution (see figure 2.1 for an illustration), formally, given a state s and an action a , then a probability can be assigned to the set of states $s' \in \{s_1, s_2, \dots\}$, such that:

$$P(s'|s, a). \quad (2.5)$$

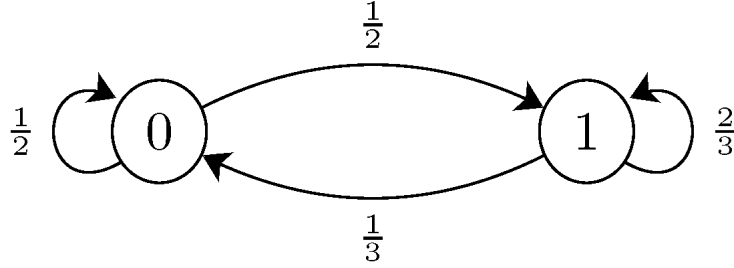


Figure 2.1: Showing the MDP's transitional probability distribution for an arbitrary action under two different states, e.g if the action is to transition to the next state from the current state $0 \rightarrow 1$ or $1 \rightarrow 0$, then the displayed probabilities indicate the rate of success of that action. Different probabilities apply to different actions, i.e action: $0 \rightarrow 0$ or $1 \rightarrow 1$ may yield different probabilities under the same MDP.

Furthermore, simple search algorithms e.g (breadth and depth first search algorithms [27, 28]) that provides a set of actions to take given a state, will fail in a environment that is governed by a Markov decision process over many iterations, this is due to those algorithm's inability to deal with the stochastic nature of those environments, therefore we use a decision policy as a solution.

A policy is a function of states that suggests an action to take at time t within any state s , thus, at time $T = t - 1$ and state s_{t-1} we write:

$$\pi_t(a_{t-1}|s_{t-1}) = P(A_{t-1} = a_{t-1}|S_{t-1} = s_{t-1}) \quad (2.6)$$

The output of the policy is a probability distribution over a possible number of actions, hence $\pi_t(a_{t-1}|s_{t-1}) \in [0, 1]$ and $\sum_{a_{t-1}} \pi_t(a_{t-1}|s_{t-1}) = 1$; the outputs must be constrained between 0 and 1, and must be normalised. Furthermore, the objective is to find the policy that provides us with the best expected return $\mathbb{E}[G_{1 \rightarrow T}]$, this policy is called the optimal policy and given the symbol π^* , formally:

$$\pi_t^*(a_{t-1}|s_{t-1}) = \underset{\pi}{\operatorname{argmax}} \mathbb{E}[G_{1 \rightarrow T}] \quad (2.7)$$

The actions provided by the policy are computed live as the agent plays the environment, meanwhile, the aforementioned search algorithms computes the whole trajectory first, and thus is likely to fail due to the stochastic nature of the environment. Furthermore, an optimal policy π^* will have a higher chance of success within the environment because it computes the trajectory step by step as the agent plays the game, therefore the decisions it makes are live, meaning that the agent's actions can be repeated and corrected given the stochastic nature of the environment.

Once the policy provides the probability distribution over the available actions, the agent proceeds by sampling from said distribution, a key concept that follows is the method of sampling from this distribution; we can either sample the mean of the distribution or perform random sampling given the distribution's parameters, this is the paradigm of knowledge exploitation and exploration respectively [25], where in the former the agent is "greedy"/"exploitative" and selects the action that will most probably yield the best outcome (according to the policy), while the latter is explorative and the agent/policy tends to select a sub-optimal action within the given state, allowing the agent to possibly find a new optimal action or sequence of actions that it previously thought to be sub-optimal. The balance between the rates of exploitation and exploration is key, as having one without the other is shown to yield a sub-optimal policy [29], therefore in this investigation we apply an exploitative policy; a stochastic policy, that explores by sampling from a normal distribution, thus, it naturally exploits it's knowledge sometimes, while also exploring other times (see chapter 5).

In the following chapter we introduce an important and state of the art machine learning paradigm, that of actor critic and policy gradients methods.

Chapter 3

Actor Critic Methods

3.1 Actor Critic

Actor critic methods employ a pair of inference models working in parallel [25], the actor is identical to the policy (just a naming convention used interchangeably), and similarly the critic is referred to as the value function.

The actor's role is to select/sample an action given any state S_t , this is done by carrying out a forward pass of our network, where features are obtained from the feature extractor, and then the actor network uses those features to output a probable action, given this action we can then pass it to the environment to obtain a new state and a reward, this process is repeated and the relevant data is stored in a replay buffer, in order to be used in the optimisation algorithm (section 6.6). Similarly to the actor, the critic's role is to take in the same state, it then computes a value of the state, this value is then used to calculate critic's loss, and is added as a component to the actor's loss, doing this reduces the variance of the actor's loss function and allows the actor to learn efficiently, at the same time the critic itself improves by outputting state values that are more accurate, this is because the critic's update step is a supervised learning problem, where the value calculated by the state is compared in a Mean Square Error (MSE) loss function to the rewards/values obtained by the actor when playing the game [25]), thus resulting in a harmonious relationship, where both the actor and the critic depend on each other in

order to improve their respective performances.

In our investigation the actor and the critic are both deep neural networks, however, they don't always have to be as they could be any simpler machine learning models. Given that we're using deep neural networks and that they are parameterised models, it is then required for us to find the best parameters by training both models simultaneously, with the goal that the parameters converge to the optimal policy π^* . It's key to note that both the actor and the critic share a fundamental feature extractor (figure 3.1), given that our environment is visually based, we need to extract features using a CNN, in order to be able to efficiently teach our models. This is further broken down in chapter 5.

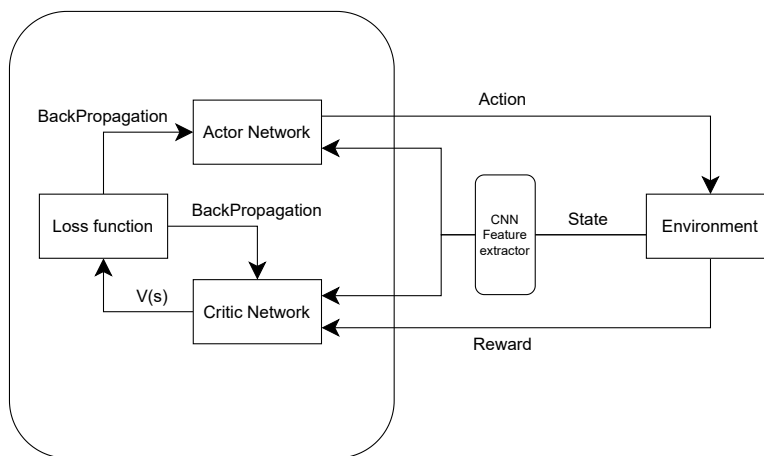


Figure 3.1: Showing how the actor critic model works and how it interacts with the environment, the loss function block includes the value function's loss which is used to further calculate the error of the actor network and to update the critic network. (Note: The PPO algorithm and loss function is further broken down in figure 6.3)

3.2 Policy Gradients

Policy gradient methods are techniques that involve parameterised policies and their respective gradients with respect to a loss function.

Before the advent of policy gradient methods, in order to be able to play an environment successfully we would have had to utilise action-value methods to craft a policy [25], where action selection was based on a learned estimate of the value for each action

within a given state (Note: to select an action the policy "consults" the value function), however, policy gradient methods do not depend on a learned value in order to select an action, instead the action selection function (the policy) is separated from the value function entirely, thus allowing us to select actions without the need to estimate values during inference, still, we use a value function in order to train our policies in an actor-critic style as described in section 3.1. In policy gradient methods, the parameter vectors of the policy and value function are given the symbols $\boldsymbol{\theta}$ and $\boldsymbol{w}$ respectively, therefore, for our parameterised policy and value functions we can write:

$$\pi(a|s, \boldsymbol{\theta}) = P(A_t = a | S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}) \quad (3.1)$$

and

$$\hat{v}(s, \boldsymbol{w}) = \mathbb{E}_\pi [G_t | S_t = s, \boldsymbol{w}_t = \boldsymbol{w}] \quad (3.2)$$

Following this, we would now want to update the policy parameters in order to obtain the optimal policy π^* , we do this by computing the gradient of a loss function $J_\pi(\boldsymbol{\theta}_t)$ with respect to the policy parameters, we then apply a discount and update the parameters, formally, we compute the following:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha_\pi \nabla J_\pi(\boldsymbol{\theta}_t) \quad (3.3)$$

where $\boldsymbol{\theta}_{t+1}$ are the new parameters, $\boldsymbol{\theta}_t$ are the old parameters, α_π is the discount constant, ∇ is the gradient operator and $J_\pi(\boldsymbol{\theta}_t)$ is the policy loss function with respect to the parameters. On the other hand, we also update the parameters of the value function $\boldsymbol{w}$, such that:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha_v \nabla J_v(\boldsymbol{w}, R_{t:T}) \quad (3.4)$$

where $\boldsymbol{w}_{t+1}$ are the new value function/network parameters, $\boldsymbol{w}_t$ are the old parameters, α_v is the discount constant, ∇ is the gradient operator, $R_{1:T}$ are the rewards given by the environment between time steps t and T and $J_v(\boldsymbol{w}_t, R_{1:T})$ is the value loss function with respect to the value function parameters and the rewards. In this investigation we use the Proximal Policy optimisation algorithm (PPO), which is a state of the art policy gradient algorithm that utilises a value function. (Note: the value computed by the parameterised value function is used in the policy loss function $J(\boldsymbol{\theta}_t)$ through the Generalised Advantage Estimation values (GAEs) [23], this is further discussed in section 6.6.)

In chapter 4, we proceed by introducing Tensors; Tensor Networks and their properties; and the types of Tensor Networks used in our investigation.

Chapter 4

Defining Tensor Networks

4.1 Definition Of A Tensor

A Tensor is a generalisation of the common linear algebra methodology used to organise a system of linear equations or numbers, this includes scalars, vectors and matrices, for example, a zero Tensor is a rank 0 Tensor that is equivalent to a scalar, a 1 Tensor (rank 1) corresponds to a vector, a 2 Tensor (rank 2) to a matrix and a 3 Tensor (rank 3) to a 3D matrix,

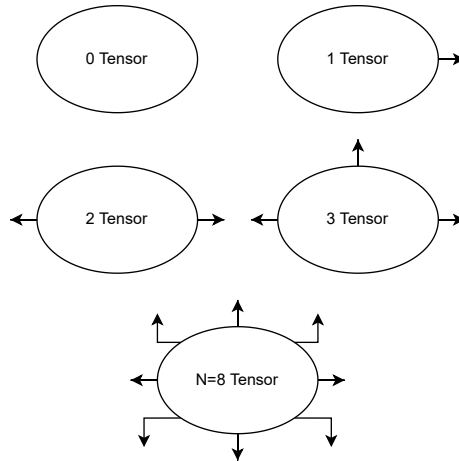


Figure 4.1: Showing the different types of Tensors with a general example where N is the total number of arrows, with each arrow representing a dimension of an arbitrary size, e.g where $N = 3$ a Tensor can have dimension sizes (16, 8, 32).

therefore, an arbitrary N-Tensor represents an N dimensional array, and in order to access a scalar value within such a Tensor, N indices are needed, formally:

$$A = A_{ijk} \quad (4.1)$$

where A is a 3 Tensor with indices $\{i, j, k\}$. It's key to note that in our implementation Tensors do not represent a system of linear equations, but instead, they are simply used as a convenient way to store machine learning model parameters, this allows us to leverage the key (organisation and manipulation) properties of Tensors and Tensor Networks to construct novel architectures for use as machine learning models.

4.2 Definition Of A Tensor Network

When obtaining the dot product between two matrices A and B, it is a requirement that their "contracting" indices match, in other words the last dimension of the first matrix A and the first dimension of the second matrix B must be equal in length, the result is another 2D matrix.

$$C_{xy} = AB = \sum_i A_{xi} B_{iy} \quad (4.2)$$

To generalise this to a pair of Tensors, the same requirements apply, however, the result will have a specific number of dimensions that is dependant on the dimensions of the original Tensors, and we change terminology to refer to this new operation as a Tensor contraction, this is due to the elimination of two dimensions (one from each Tensor), and can be written as:

$$C_{zyxjk} = AB = \sum_i A_{zyi} B_{ixjk} \quad (4.3)$$

Where C is an $(N+M-2)$ rank Tensor after contraction, A and B are two Tensors of ranks $N = 3$ and $M = 4$ respectively, and $\{x, y, z, i, j, k\}$ is a set of indices indicating distinct dimensions (Note: the following: A_i and B_i , indicates two Tensors of an i dimension of the same size). Alternatively this contraction operation can be visualised as follows:

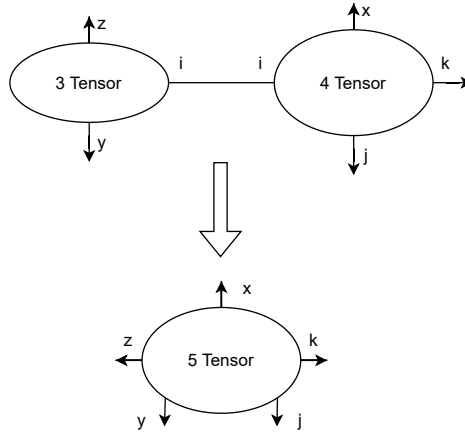


Figure 4.2: Showing an example of a Tensor contraction, where an $N=3$ Tensor is contracted with an $M=4$ Tensor to produce a $(N+M-2)$ order Tensor; a 5 Tensor.

Furthermore, another operation that we leverage is the trace, for a 2D matrix (a 2 Tensor) this is formally defined as:

$$\text{tr}(A) = \sum_{i=1}^n a_{ii} \quad (4.4)$$

where tr is the trace operator, A is an arbitrary square matrix of size $n \times n$ and a_{ii} denotes the entry to the i th column and row. This operation produces a scalar value and is equivalent to **a)** in figure 4.3, similarly between two Tensors, mirroring the visualisation **d)** in figure 4.3 we write:

$$\text{tr}(A^T B) = \sum_{i=1}^m \sum_{j=1}^n a_{ij} b_{ij} \quad (4.5)$$

where A and B are two matrices of sizes $m \times n$, (Note: We transpose A so the aforementioned dimension size condition is met.)

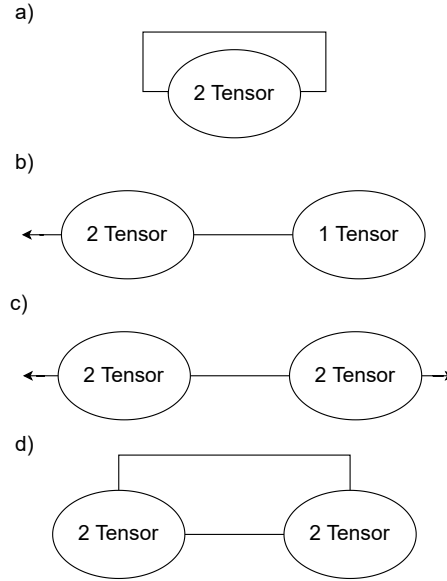


Figure 4.3: **a)** showing the result of the trace for a 2 Tensor (Eqn. 4.4), this produces a scalar value. **b)** showing the result of a contraction between one of the indices of a 2 Tensor and the only index of a 1 Tensor, producing a 1 Tensor as a result (Eqn. 4.3). **c)** showing the contraction between two 2 Tensors along only one of their indices producing a 2 Tensor (Eqn. 4.3). **d)** showing the trace operation between two 2 Tensors where we contract along both indices, resulting in a scalar value (Eqn. 4.5). (Note: **d)** is identical to **a)**, due to the dimension reduction effect of the contraction operation and the fact that the trace is a contraction operation.)

Given figure 4.3 and the similarities between the trace and a pair regular contractions between two Tensors, we can say they're indistinguishable, and we therefore treat the trace as just another (stacked) contraction operation. Those two operations, form the basis for many Tensor Network architectures, in section 4.3 we proceed by introducing the matrix product operator and the matrix product state, which are our implemented Tensor Network architectures.

4.3 Applied Tensor Networks

The two types of Tensor Networks used in our machine learning implementation are the matrix product operator (MPO) [30, 31] and the matrix product state (MPS) [32], for the policy and value functions respectively (see chapter 6). The matrix product operator is a finite set of rank 3 and 4 Tensors, where 1 or 2 indices connect the neighboring Tensors (intra-bond) and the other 2 are left free (inter-bond and output), we can represent this simply as:

$$M_{s'_1, s'_2, s'_3 \dots s'_n}^{s_1, s_2, s_3 \dots s_n} = \sum_{\alpha} A_{s'_1}^{s_1 \alpha_1} A_{\alpha_1 s'_2}^{s_2 \alpha_2} A_{\alpha_2 s'_3}^{s_3 \alpha_3} \dots A_{\alpha_{n-1} s'_n}^{s_n} \quad (4.6)$$

where M is the matrix product state; A is a an arbitrary 3 or 4 Tensor (depending on position); s_n and s'_n represents free indices for the inter-bond and output respectively; and α indicated the intra-bond index [33]. Additionally, this can be represented using graphical Tensor notation as follows:

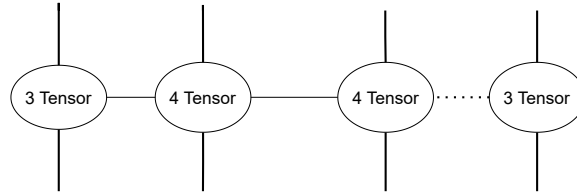


Figure 4.4: Showing an example MPO; a finite set of 3 and 4 Tensors, with neighboring Tensors contracted along a contraction dimension (intra-bond), where the first and the last Tensors have only a single neighbor and are the only 3 Tensors, the diagram also shows the two dimensions left uncontracted, for the inter-bond and output (Note: This can be extend so all Tensors are rank 4 Tensors).

The requirements as previously mentioned in this chapter have to be met, this allows for the contractions shown in figure 4.4, furthermore, the two dimensions left free are the inter-bond and output dimensions, the intra-bond dimension allows us to connect the Tensor Network to another Tensor, in our case the feature extractor, and when the two Tensors are contracted we will be left with a single dimension for each agent, the output dimension, this is used to sample an action.

On the otherhand the MPS is composed of ranks 2 and 3 Tensors depending on

position, this is written as:

$$T^{s_1, s_2, s_3 \dots s_n} = \sum_{\alpha} A_{\alpha_1}^{s_1} A_{\alpha_1 \alpha_2}^{s_2} A_{\alpha_2 \alpha_3}^{s_3} \dots A_{\alpha_{n-1}}^{s_n} \quad (4.7)$$

where T is the MPS Tensor, A is either a rank 2 or 3 Tensor, s_n is the inter-bond index and α_n represents an intra-bond index, thus resulting in an output that is a zero tensor; a scalar value, furthermore, Eqn. 4.7 can be represented using graphical Tensor Network notation as follows:

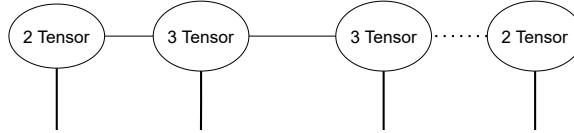


Figure 4.5: Showing an examples MPS: a finite set of 2 and 3 Tensors, with neighboring Tensors contracted along a contraction dimension, where the first and the last Tensors have only a single neighbor and are the only 2 Tensors, the diagram also shows one dimension left uncontracted representing the inter-bond index, later used to contract the MPS with the feature vector (Note: This can be extend so all Tensors are rank 3 Tensors).

This MPS is contracted with the feature extractor, the result of this contraction is the output of the value function; the value of the state, which is used in our policy gradients algorithm (actor critic see sections 3.1, 6.6).

It should now be clear how we utilise Tensors and Tensor contractions to construct Tensor Networks. In later chapters, we construct an MPS and MPO to use as the heads for our value and policy functions, we then connect and contract our Tensors (MPS and MPO) to the CNN feature extractor, this output is used to calculate the shared value of the multi-agent state, where we see the first implementation of the trace operation; and to compute the unique actions per agent in the policy function (see figure 6.2 and chapter 6). Nonetheless, in the next chapter we introduce the problem setting and the baseline solution for multi-agent reinforcement learning.

Chapter 5

Problem Setting And Baseline Solution

5.1 Petting Zoo

We will be using Petting-Zoo as our environment provider, this library offers many multi agent environments, and for our investigation we will use the piston-ball environment. The Petting zoo api allows us to setup and interact with our environment in two different ways, firstly as a sequential environment; where the actions for each agent are processed independently, and secondly as a parallel environment; where the actions for all agents are processed at the same time, we utilise both modes to evaluate and train our models.

5.2 Pistonball

The piston-ball environment as shown in Figure 5.1, is a simple physics based environment, where the task is to move a ball that spawns on the right side of the screen all the way to the left wall, and intuitively, this task can be achieved by a coordinated and well timed set of actions by pistons. Each piston has the ability to move up or down by 4 pixels per frame, therefore for optimal performance, we expect the agents to learn when to go down and when to go up, this depends on the agents ability to correctly view the environment and then to effectively learn the required behaviour.

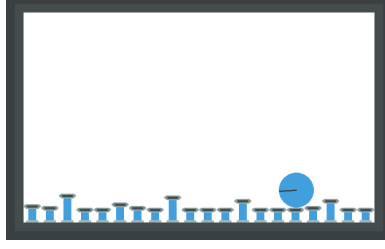


Figure 5.1: Showing a snapshot of the Piston ball environment produced by Pettingzoo, with 20 agents represented by the number of pistons and a ball (blue circle) initially spawned at the right hand side of the environment.

5.3 Environment Reward Mechanics

Each agent gets either a local or global reward, the local reward is only calculated for each piston if it is directly below any part of the ball, meanwhile the global reward is given to each agent when no part of the ball is above it. Furthermore, the rewards are based on a time penalty and the movement pattern of the ball, if a time step passes then a constant negative reward is added as a component to the global reward function; if the ball moves to the left on the x-axis then a positive reward is calculated; and if it moves to the right then a negative reward is calculated instead, where the function and scale of the reward is dependant on if it's a local or global reward, in simpler terms, for the local and global rewards we write:

$$R_t^L = 0.5\nabla x \quad (5.1)$$

$$R_t^G = \frac{100\nabla x}{x_0} + P \quad (5.2)$$

where R_t^L is the local reward at time t , R_t^G is the global reward at time t , P is the time penalty with a default value of -0.1 , and x is the position of the ball with x_0 and x_f representing the initial and final positions of the ball such that $\nabla x = x_f - x_0$. Finally the reward function given to the i^{th} agent:

$$R_{ti} = r_i R_{ti}^L + (1 - r_i) R_{ti}^G \quad (5.3)$$

where r_i is a binary environment parameter that determines if the ball is in the local region of the i th agent or not, thus resulting in a conditional reward function between the local and global rewards.

5.4 Preprocessing

Before the baseline solution is implemented we preprocess the environment such that we retain key information, since the machine is the observer, a lot of the qualities of the image such as the color or size of the image are not required for this particular task (moving the ball to the left), we therefore use a library; super suit [34] in order to achieve this, super-suit is a wrapper that sits around the petting-zoo environment, allowing us to execute preprocessing operations before the agent sees the observation.

We preprocess the observations of the environment such that each agent’s observation is in greyscale, rescaled to 84 by 84 pixels (a popular convention started by DeepMind [35]), and each observation is a stack of n time steps where n is a hyperparameter defaulting to 3, this gives us a final observation of dimension (84,84,3) for each agent out of M agents, thus resulting in shape (M -agents,84,84,3) for a single environment observation.

Furthermore, we need to construct a baseline model and train it to obtain a solution, such that there is a clear performance target for our novel model, this serves as an agent which we aim to carry out a comparative analysis against, see section 5.5 for the model architecture of choice and section 5.7 for the quantitative measure we aim to extract from the baseline model.

5.5 Baseline Solution

To establish a baseline performance we use the a very basic implementation in stable baselines3 [21], this specific implementation utilises the Nature CNN architecture as implemented in [36], followed by two separate Feed Forward networks that represent the actor and the critic, in this model architecture, each agent shares their parameters with every other agent, for simplicity, we can think of this as each agent taking their turn to

run inference using a single set of parameters.

The Nature CNN consists of 3 convolutional layers, with valid padding and a stride value of 1, with each layer being followed by a Relu activation function, this is followed by a flattening layer that outputs a feature vector of shape 512, figure 5.2, shows the general architecture used to solve Pistonball environment.

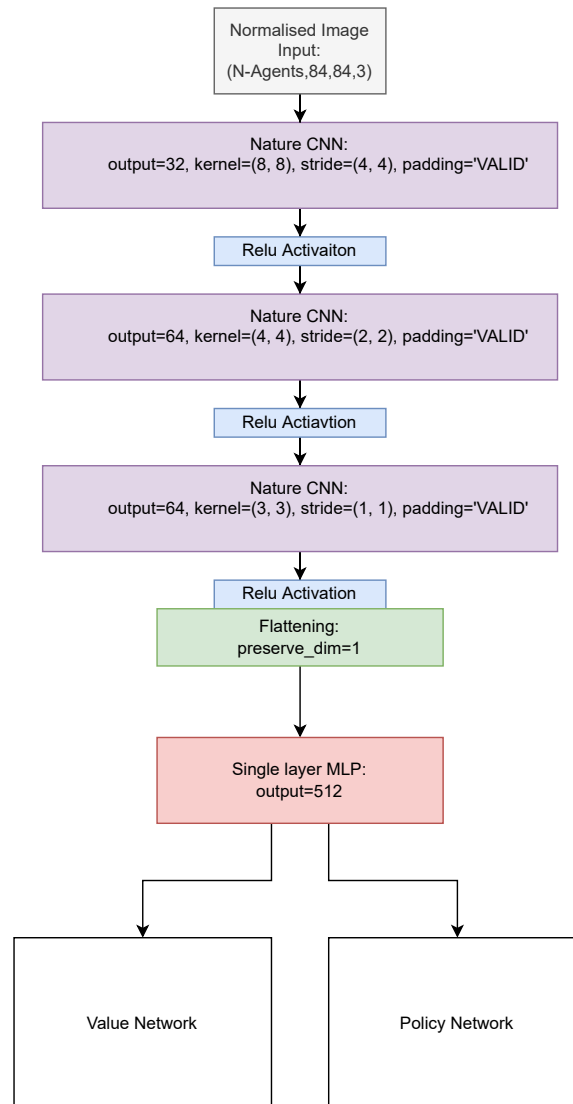


Figure 5.2: Jax-Haiku implementation of the Nature CNN, with the output of each layer acting as the input dimension to the next, followed by two separate arbitrary networks for the value and policy respectively, this represents a general model architecture.

To complete the baseline solution, we implement a pair of feed forward networks

of a single layer each, that represent the value and policy networks as shown in the general architecture in figure 5.2, with input dimensions 512 taking in the output of the feature extractor and an output dimension of size 1, representing a value and the mean of a distribution for the critic and the actor respectively (see section 3.1). The actor combines the mean with another tuneable parameter representing (the logarithmic standard deviation) in order to define a normal distribution, from which we can sample the action, this is because we would like the model to explore the environment and thus converge to a good policy (section 2.2), on the other hand, the critic outputs a state value which is used to calculate the loss for both the actor and the critic. Figure 5.3 visualises our baseline head implementation. (Note: We utilise the logarithmic standard deviation because this allows it to be any real value , thus, allowing for fluid learning [23, 37])

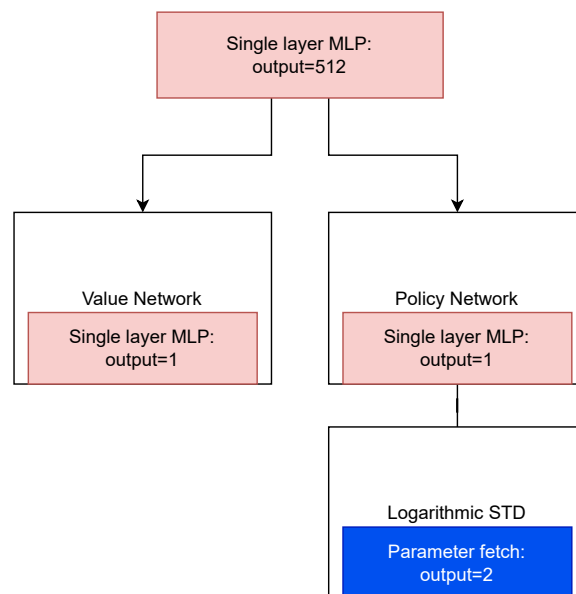


Figure 5.3: Jax-Haiku implementation of the baseline solution, taking the final output of the Nature CNN as input into the value and policy networks, each of which is a single layer MLP with output vector shapes 1 and 1 respectively, with the policy network being followed by a layer that simply fetches a new tuneable parameter, the logarithmic standard deviation, resulting a two parameter output.

5.6 PPO configurations

The following table shows the hyperparameters used to train the baseline model, furthermore, the results of training can be found in chapter 7.

Hyperparameters	Value	dtype
seed	0	float
entropy coefficient	0.01	
value function coefficient	1.0	
learning rate	0.0001	
max gradient norm	0.9	
discount	0.99	
clips epsilon	0.2	
GAE Parameter	0.95	
Max agent steps	2_000_000	int
Buffer size	1025	
Epochs	7	
Batch size	128	

Table 5.1: Showing the different hyperparameters used to train the baseline model.[21, 23]

5.7 Quantitative Measure

To quantify the performance of the baseline model we will use the expected return as the primary measurement when the policy is used as a deterministic policy, this will switch the model from an explorative model to an exploitative model, such that it picks the best known action to it after training with exploration, we also evaluate the optimal policy as a non-deterministic policy and we compare both to a random policy.

In chapter 6 we detail the architecture of the Tensor Network model, followed by a visualisation and an execution of a forward pass, and finally in section 6.6 a complete analysis is carried out to compare the baseline model with the Tensor Network model.

Chapter 6

An Implementation Of Tensor Networks For MARL

In this chapter we introduce the main body of novel research, having had the background explained and the baseline model setup in the previous chapters, it is now feasible to begin the implementation.

6.1 Why Jax ?

Jax is an emerging Machine learning framework [20], and it offers many advantages such as Just In Time compilation (JIT), automatic differentiation and automatic vectorisation; allowing for acceleration, intuitive & stackable gradient calculations and parallel computation mappings respectively. However, the primary feature of Jax that gravitates us to its use in novel research is its direct mirroring to Numpy [38], meaning that we can run code that is identical to Numpy in format, directly on the GPU. The advantages of this, is that research is accelerated not only in runtime speed but also in implementation speed; it is easier to implement novel research (mathematically) directly on Jax-Numpy when compared to using other machine learning libraries (PyTorch or TensorFlow [39, 40]), due to this, Jax isn't only limited to research in machine learning but general mathematics making all Numpy based research GPU enabled [41, 42, 43].

6.2 Complete Architecture And Visualisation

The model architecture is relatively simple to detail out since the MPO and MPS networks have been explained in chapter 4, simply, we take the output of the feature extract and compress it's dimensions using a single layer MLP with an output of 64, this is done to ease computational costs since the number of parameters scales non-linearly with both the Inter and Intra-Bond dimensions sizes (see section 7 for experimental proof), we then apply a LogSoftmax activation function after the output has been compressed, this has been shown to work best when working with Tensor Networks for finite Markov decision processes [18], as it normalises the compressed output, thus offering numerical stability for operations done by the Tensor Network. Furthermore, a branch now formulates, where the output of the LogSoftmax is passed to the value and policy networks, each of which is a Tensor Network with contraction/Intra-Bond dimension size of 32 and 32 and output sizes of 1 and N-actions, where N-actions = 3 for the discrete case of our environment of choice (chapter 5). Figure 6.1 is a high-level visualisation of the architecture.

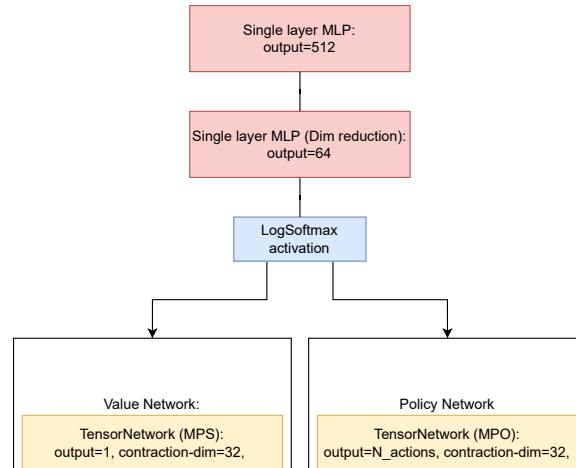


Figure 6.1: Jax-Haiku implementation of a Tensor Network solution, taking the final output of the Nature CNN and applying a dimension size reduction to $output = 64$ followed LogSoftmax activation, before feeding the 64 feature vector into the value (MPS) and policy Networks (MPO), each of which is a Tensor Network with output shape and contraction dimension sizes of 1,N-actions and 32,32 respectively.

Furthermore, each of the pair of Tensor Networks is contracted as mentioned previously

with the output of the LogSoftmax activation, however, instead of having rank 2 or 3 Tensors left at the edges of the each of the networks (Fig. 4.4), we instead make the whole MPS and MPO a set of rank 3 and 4 Tensors respectively, the extended Tensors will have two extra indices uncontracted, to work around this we contract those two indices/dimensions using a trace operation, this leaves us with the expected Tensors for the value (1 by rank 0) and policy networks (N by rank 1), see figure 6.2.

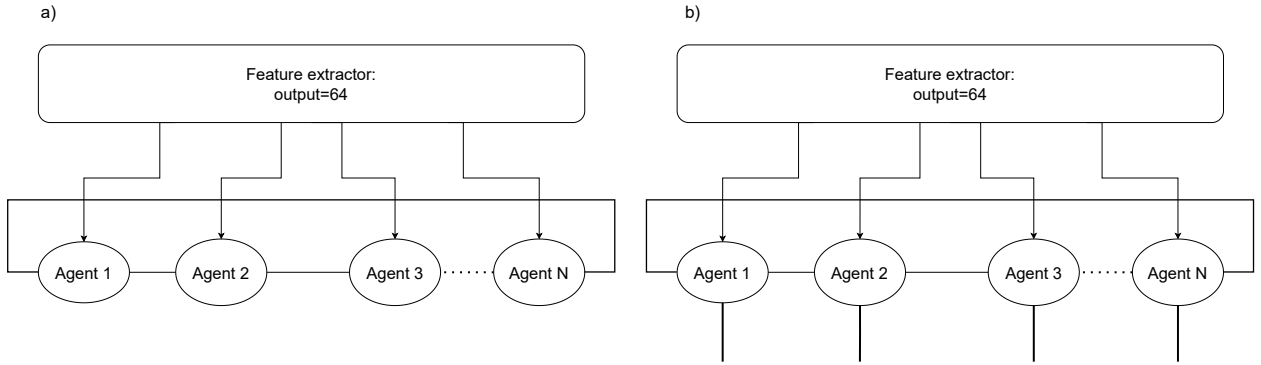


Figure 6.2: **a)** Showing the resultant (rank 0) Tensor of the MPS contraction with the feature extractor output. **b)** Showing the resultant (rank 1) Tensor of the MPO contraction with the feature extractor output. (Note: In practical implementation, we do not store a tensor of N dimensions/indices, instead, a compact Tensor that represents this larger Tensor, e.g for **b)** we store a rank 2 Tensor of shape (N, 3), for N-agents and 3 possible actions (no-action, down, up).)

To compute the value of the state and sample actions from each of the Tensors, we carry out specific calculations for each part of the network, this is demonstrated in sections 6.3 and 6.4 respectively.

6.3 The Value Head: State value

To obtain a state value, we must obtain:

$$v(s) = T_{s_1} T_{s_2} T_{s_2} \dots T_{s_n} \quad (6.1)$$

where $v(s)$ is the state value function, T_{s_n} is a (rank 2) previously a (rank 3 Tensor [MPS]) contracted with the state s_n ; agent wise state from the feature extractor.

6.4 The Policy Head

In order to sample actions for each of the agents, we must obtain the policy, this is done using the following expression:

$$\pi(a|s) = \frac{\phi_s(a)^2}{\sum_a \phi_s(a)^2} \quad (6.2)$$

with $\phi_s(a)$ taking the form:

$$\phi_s(a) = M_{a_1 s_1} M_{a_2 s_2} M_{a_3 s_3} \dots M_{a_n s_n} \quad (6.3)$$

and the denominator of 6.2 being the normalisation of the MPO, formally computed by:

$$N(s) = \sum_a \phi_s(a)^2 = \left(\sum_a M_{a_1 s_1} \otimes M_{a_1 s_1} \right) \left(\sum_a M_{a_2 s_2} \otimes M_{a_2 s_2} \right) \left(\sum_a M_{a_3 s_3} \otimes M_{a_3 s_3} \right) \dots \left(\sum_a M_{a_n s_n} \otimes M_{a_n s_n} \right) \quad (6.4)$$

Where π is the policy or MPO, conditioned on the state, and giving an action for each agent, where, if we would like to sample an action we use the chain rule, such that:

$$\begin{aligned} \pi(a|s) &= \pi(a_1 a_2 \dots a_n | s) = P(a_1 | s) P(a_2 | a_1, s) \\ &\quad P(a_3 | a_1, a_2, s) \dots P(a_n | a_1, a_2, \dots, s) \end{aligned} \quad (6.5)$$

To obtain that we calculate the probability distribution of the first actor, and insert that into the calculation of the probability distributing of the following actor and so on:

$$P(a_1 | s) = \sum_{a_1 a_2 a_3 \dots a_n} \pi(a|s) = \frac{1}{N(s)} \sum_{a_2 a_3 \dots a_n} \phi(a)^2 \quad (6.6)$$

where $P(a_1 | s)$ is the probability distribution of actions for the first actor.

6.5 Executing A Forward & Backwards Pass

Finally to execute a forward pass on an observation of our environment, we create a Haiku class that represents the Tensor Network architecture described in the previous sections, such that the Inter-Bond dimension, output dimension size, and class of the

Tensor Network are configurable layer parameters; and by manipulating those parameters, we can easily create an MPO or MPS with specific output and Inter-Bond dimension size. Furthermore, after specifying the layer, we simply couple them in a function with the feature extractor, such that the input to the function are the observations, and the output of the feature extractor is used as input for both the value and policy Tensor MPS and MPO respectively.

This results in a successful forward pass, and by extensions a successful backwards pass since the latter is automatically computed using Jax’s grad function, however in order to successfully train the network, we need to use a reinforcement learning algorithm, we attempt adapt the PPO algorithm, such that it can handle a Multi-Agent shared policy, and thus carry out a useful backwards pass.

6.6 Modifying PPO

Reinforcement learning algorithm are very unstable due to the extreme jumps in the action space [23], thus resulting in agents that seem to be learning really well and obtaining good performance before crashing completely, PPO applies limitation to the movement of the model parameters such that this translates to a controllable movement in the action space [23], by applying gradient clipping and updating the parameters based off of a loss function that compares the new and old policy.

Figure 6.3 shows how the PPO algorithm is structured, with the highlighted blocks indicating where modifications were made.

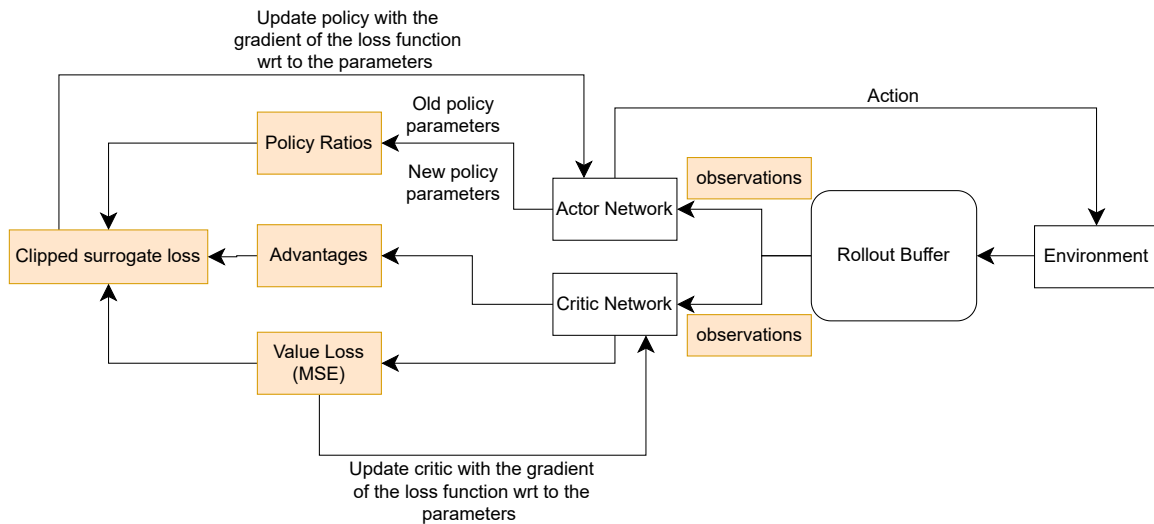


Figure 6.3: Showing the actor-critic PPO algorithm outline, including the relation between the environment, policy and value networks through the RolloutBuffer, it also shows the intermediate values (policy ratio, Advantages and Value loss) used to calculate the surrogate loss and the parameter update step for the value and loss function, with the highlighted (orange) blocks indicating the location of modifications made to the algorithm, to allow Multi-Agent Shared Policies to successfully calculate the loss for each agent.

In simple terms, those modifications attempt to generalise the operations of PPO, such that they're viable for an extra dimension; the agent dimension, thus, along with the observation dimension and image specification we expect the input shape to be the following: (observation, agent, 84, 84, 3) see section 5 for specifics. Algorithm 1 details out the specific modifications made to each part. where swapping and reshaping/combining of dimensions are the primary operations used to adapt this algorithm.

Algorithm 1 Actor-Critic PPO for Multi-Agent Reinforcement learning

Modified from Algorithm 1 in [23]

```

for iteration=1,2,3... do
  for timestep=1,2,3...T do
    Run policy  $\pi_{\theta_{old}}$  for all agents
    Store variables into memory with shape -> (observation, N-agents,...)
  end
  Fetch variables from memory/rollout buffer
  Flip dimensions of variables to (N-agents, observation,...)
  Calculate agent wise Advantages separately for each agent
  Flip dimensions of variables to (observation,N-agents,...)

  Create dataframe -
  (state, values, logprob, rewards, dones, next states, GAEs, targets, values)

  Batch dataframe with batch size
  Randomise order of batches
  for batch in Batch: do
    ...
    * for value in
      (values, behavior-values, actions, target-values,
      behaviour-log-probs, log-probs, entropy, advantages)
    , reshape (-1,1) *

    Calculate Policy ratios
    Calculate value loss
    Calculate clipped surrogate loss
    Optimise with surrogate loss  $L$  wrt to  $\theta$  for K epochs
     $\theta_{old} \leftarrow \theta$ 
  end
end

```

Chapter 7

Discussion & Results

In this chapter we introduce the results obtained during this investigation, starting with the baseline training, followed by experimental results of the scaling of our Tensor Network models and finally ending with the results of modifying PPO.

7.1 Baseline Training Summary & Tensor Network Learning Hypothesis

7.1.1 Baseline Training Summary

Table 7.1 shows the summary statistics obtained after training the baseline model, we observe that average score is highest for the deterministic policy with a very small standard deviation suggesting that the policy is able to consistently obtain a score around the mean, furthermore the non-deterministic policy performs worse than the deterministic policy and with a significantly higher standard deviation than both the contending policies, finally the random policy (the control) performs the worst, with the lowest average score, confirming that the deterministic and non-deterministic policies indeed learned to play the environment successfully.

Score Statistics	Deterministic	Non-Deterministic	Random (non-trained)
AVG. Score	1965.3 $\pm$ 8.6	1092.8 $\pm$ 419.0	-72.2 $\pm$ 262.2
min score	1942.0	247.6	-428.8
25%	1960.0	751.2	275.8
50%	1966.0	1123.7	113.5
75%	1972.0	1446.6	57.8
max score	1988.0	1810.0	1020.4

Table 7.1: Showing summary statics of the total reward obtained per episode over 100 episodes, including the Means, Standard deviations, minima, maxima, and quarterlies for the deterministic, Non-deterministic and random policies of the baseline model.

7.1.2 Tensor Network Learning Hypothesis

Conditioning on the known capabilities of Tensor Network (parameter sharing and conditional sampling), and the successful training in a controlled manner (same algorithm, hyperparameters, training steps etc), we hypothesise that when compared to the performance of the baseline model shown in table 7.1, the Tensor Network would perform better on the summary statistics shown for both the deterministic and non-deterministic with similar results to the control/random policy, this hypothesis is supported by the nature of the problem, intuitively, it's reasonable to think that if the agents are capable of communication between each other, then their overall performance will improve as their actions could be a lot more coordinated, furthermore, if there is no performance gain, then measuring the time that it takes for the model to reach the same performance as the baseline could be another avenue for proof of the capabilities of Tensor Networks, this is done by comparing the learning curves of both the baseline model and the Tensor Network model, nonetheless, this remains an open problem.

7.2 Scaling

All of the following experiments are run on a 4 core Intel CPU, utilising Jax’s JIT function where appropriate, where the control values for each for the variables as well as their testing range are shown in table 7.2.

Parameter	Control	Experimentation range
N-Agents	20	2-100
Intra-Bond dim. size	32	2-64
Image size	84	2-150

Table 7.2: Showing summary of the experimentation ranges and an outline of the controlled parameters

7.2.1 Number Of Agents

Figure 7.1 shows the relationship between the number of agent and the number of parameters for both the baseline model and the Tensor Network, we demonstrate that the baseline model’s number of parameters has no correlation with the number of agents playing the game, this is because all agents share the same parameters, however, a direct positive and linear correlation is visible between the number of parameters and number of agents for the Tensor Network.

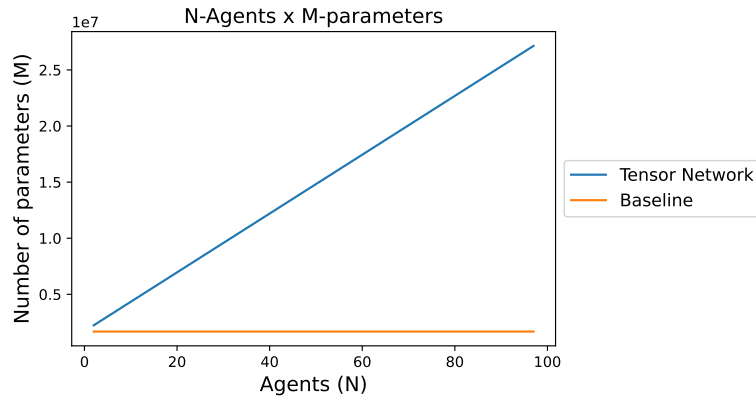


Figure 7.1: Showing how the number of parameters changes with respect to the number of agents for both the baseline model’s policy and value; and the non-shared Multi agent policy and value Tensor Networks

Furthermore we also investigate the relation between the inference time and the number of agents for both models, observing a small positive correlation and a strong positive correlation for the baseline and the Tensor Network respectively, with JIT offering a significant inference time improvement for the baseline, while giving minor improvements to the Tensor Network. This result demonstrates that to scale for the number of agents, a linear increase in the number of parameter and in inference time is to be expected with JIT not offering as much of an improvement is computational speedup (Note: No GPU was used for inference here).

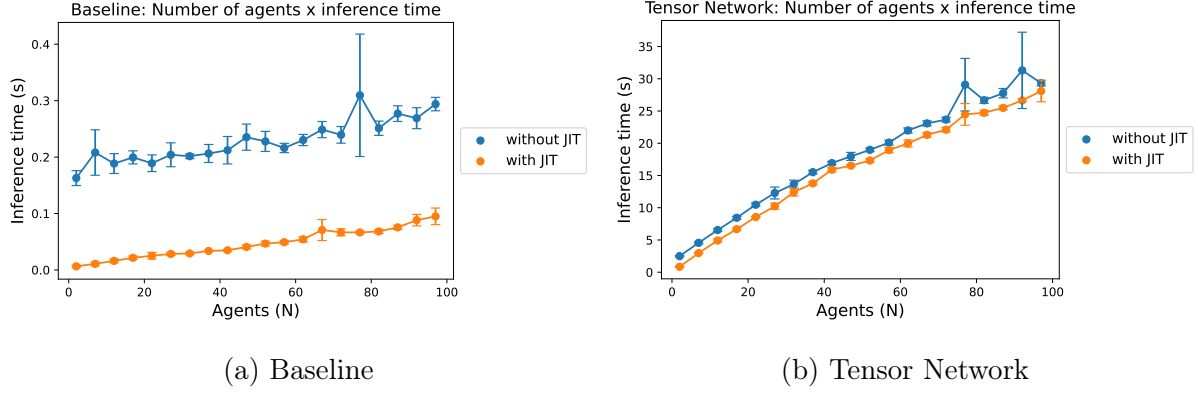


Figure 7.2: Showing how the number of agents (x-axis) effects the inference time (y-axis) with and without JIT (Just in time compilation), for both the baseline model and the Tensor Network model, where for each N-agents the measurement is repeated 10 times, and the standard deviation is represented by the vertical error bars.

7.2.2 Intra-Bond Dimension Size

Figure 7.3 shows the relationship between the Intra-Bond dimension and the Number of parameters, showing a clear non linear relationship. We also investigate (Figure 7.4) the relationship between the inference time and the size of the Intra-Bond dimension, where we observe an exponential relationship between the two variables. This suggests that Tensor Networks will not be scalable to extremely large modes of communication between the present agents.

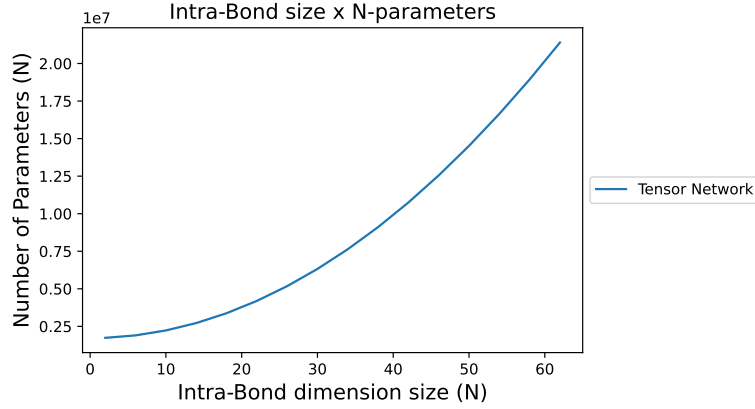


Figure 7.3: Showing how the number of parameters of the Tensor Network (MPS) scales with the Intra-Bond dimension size, where the bond dimension size ranges from 2 to 64.

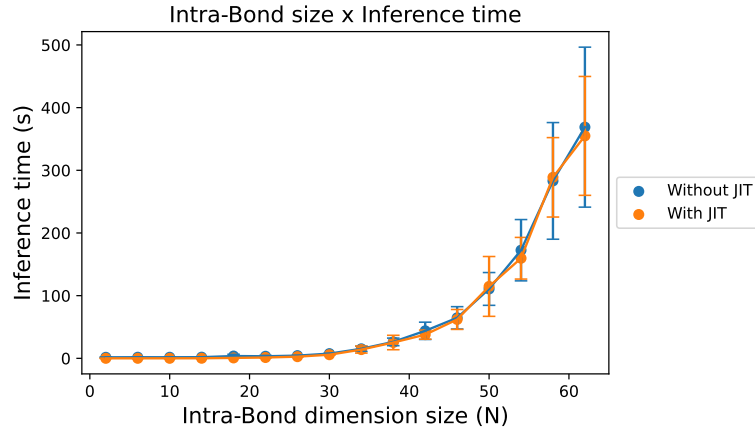


Figure 7.4: Showing how the Inference time of the Tensor Network (MPS) scales with the Intra-Bond dimension size, where the bond dimension size ranges from 2 to 64, where for each dimension size the measurement is repeated 10 times, and the standard deviation is represented by the vertical error bars.

7.2.3 Image Size

Figure 7.5, shows the relationship between the image width and the inference time, where we observe that there is no clear correlation at this range of image width for either model, except a small visible positive and linear correlation with the baseline model when using JIT. This result demonstrates that the inference time of the Tensor Network model is

independent of size of input, since this hyper parameter primarily effects the feature extractor part of the architecture up to the point where the output is compressed to the required size 64. However more importantly, this result acts as a control, validating the architecture and by extensions validating the results obtained in previous experiments.

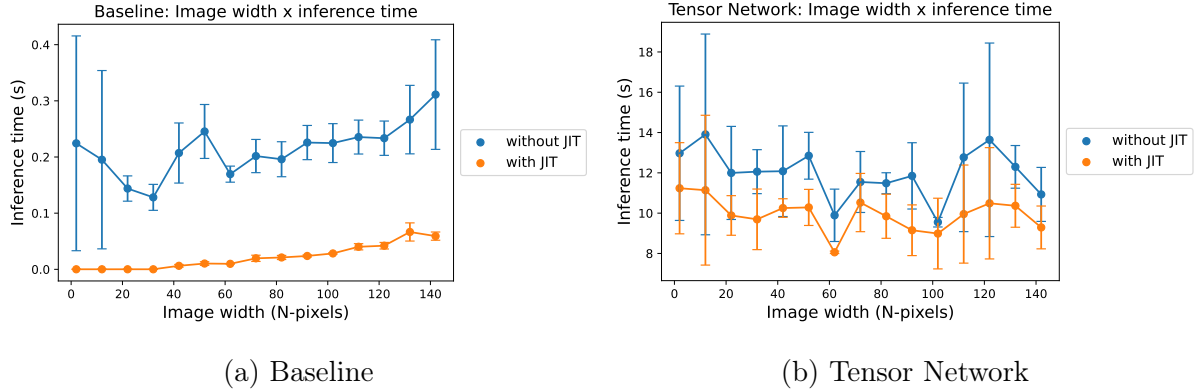


Figure 7.5: Showing how the image width (x-axis) effects the inference time (y-axis) with and without JIT (Just in time compilation), for both the baseline model and the Tensor Network using JAX, where for each image width the measurement is repeated 10 times, and the standard deviation is represented by the vertical error bars.

7.3 Learning

Carrying the changes to the PPO algorithm proposed in section 6.6, enables us to successfully carry out a backwards pass that utilises the surrogate loss function, however, the Tensor Network model still fails to learn effectively, we hypothesise that some more modifications are needed to the algorithm to account for highly technical details, possibly including; gradient calculations (freezing gradients) and hyperparameter tuning, where for the former, the loss could be calculated with the respect to a single agent at a time, this can be done using a for loop where the losses are calculate with Jax's "stop_gradient" function for all other agents, and for the latter by using a simple grid search. Nonetheless, this remain an open problem.

Chapter 8

Conclusion

In this work we began by introducing Multi-Agent reinforcement learning and related works, which introduces the problem we would like to investigating using a novel architecture. We've also discussed the environment and the Markov decision process that governs it and decision policies, we then introduced actor critic methods and policy gradients before defining Tensors and Tensor Networks. Additionally we had a short introduction into the Piston-Ball environment and show cased our baseline solution, as well as the configurations used for the PPO algorithm. Finally we detailed our implementation of Tensor Networks using Jax and DeepMind Ecosystem, before showing and discussing the finding of the investigation.

Therefore to conclude, we have successfully applied Tensor Networks to the problem of Multi-Agent reinforcement learning, by creating a novel Tensor Network layer using Haiku and Jax as our frame works, we were able to construct a complete model that utilises the Nature CNN as the feature extractor, this allowed our model to be able to observe the environment and obtain a feature vector to feed into the Tensor Network layers. On the other hand, we successfully construct and evaluate a baseline solution to the problem, this is used to evaluate the effectiveness of the model under investigation.

Upon experimentation, we found that the baseline model successfully learns to play the game, this a concrete baseline which we hypothesised that the Tensor Network agent might be able to outperform if trained, however, although we succeeded at executing both a forwards and backwards pass using Tensor Networks, we did not manage to obtain a

working PPO algorithm which the model could use to learn, therefore this remain an open problem.

However, we perform scaling experiments on the Tensor Network and compare that with the baseline model where applicable, we found that the Tensor Network scales linearly with the number of agents in inference time and number of parameters, while it scales non linearly (exponentially) with the Intra-bond dimension size for both the inference time and the number of parameters. Furthermore, we also investigate the trivial case of image size, since this had no effect on the Tensor Network’s parameters, our confidence in the results obtained increased, as this validates our model structure, where the feature extractor reduces the image into a feature vector of a specified size, before feeding passing it on to the Tensor Network.

Nonetheless, this investigation is by no means complete, for instance, further modifications to the PPO algorithm are needed in order to have it correctly working with the Tensor Network architecture proposed here, following that detailed and controlled experimentation have to be carried out in order to confirm or deny the hypothesis of efficient/coordinated learning we proposed, not to mention further scaling experimentation and improvements to the architecture for performance.

Bibliography

- [1] J. Terry, “Multi-agent deep reinforcement learning in 13 lines of code using petting-zoo,” 2022.
- [2] C. Roberts, A. Milsted, M. Ganahl, A. Zalcman, B. Fontaine, Y. Zou, J. Hidary, G. Vidal, and S. Leichenauer, “Tensornetwork: A library for physics and machine learning,” 2019.
- [3] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Physical Review Letters*, vol. 91, oct 2003.
- [4] D. Perez-Garcia, F. Verstraete, M. M. Wolf, and J. I. Cirac, “Matrix product state representations,” 2006.
- [5] G. Vidal, “Entanglement renormalization,” *Physical Review Letters*, vol. 99, nov 2007.
- [6] G. Evenbly and G. Vidal, “Tensor network renormalization,” *Physical Review Letters*, vol. 115, oct 2015.
- [7] F. Verstraete and J. I. Cirac, “Continuous matrix product states for quantum fields,” *Physical Review Letters*, vol. 104, may 2010.
- [8] S. R. White and R. L. Martin, “iab initio/i quantum chemistry using the density matrix renormalization group,” *The Journal of Chemical Physics*, vol. 110, pp. 4127–4130, mar 1999.
- [9] S. Szalay, M. Pfeffer, V. Murg, G. Barcza, F. Verstraete, R. Schneider, and Örs Leg-
eza, “Tensor product methods and entanglement optimization foriab initio/i quantum

- chemistry,” *International Journal of Quantum Chemistry*, vol. 115, pp. 1342–1391, may 2015.
- [10] J. Haegeman, T. J. Osborne, H. Verschelde, and F. Verstraete, “Entanglement renormalization for quantum fields in real space,” *Physical Review Letters*, vol. 110, mar 2013.
- [11] F. Verstraete and J. I. Cirac, “Continuous matrix product states for quantum fields,” *Physical Review Letters*, vol. 104, may 2010.
- [12] A. Milsted and G. Vidal, “Geometric interpretation of the multi-scale entanglement renormalization ansatz,” 2018.
- [13] C. Bény, “Causal structure of the entanglement renormalization ansatz,” *New Journal of Physics*, vol. 15, p. 023020, feb 2013.
- [14] B. Swingle, “Entanglement renormalization and holography,” *Physical Review D*, vol. 86, sep 2012.
- [15] E. M. Stoudenmire and D. J. Schwab, “Supervised learning with quantum-inspired tensor networks,” 2016.
- [16] Y. Levine, D. Yakira, N. Cohen, and A. Shashua, “Deep learning and quantum entanglement: Fundamental connections with implications to network design,” 2017.
- [17] Y. F. Xiao-Yang Liu¹, “Quantum tensor networks for variational reinforcement learning,” 2020.
- [18] E. Gillman, D. C. Rose, and J. P. Garrahan, “A tensor network approach to finite markov decision processes,” 2020.
- [19] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of Python+NumPy programs,” 2018.
- [20] I. Babuschkin, K. Baumli, A. Bell, S. Bhupatiraju, J. Bruce, P. Buchlovsky, D. Budden, T. Cai, A. Clark, I. Danihelka, C. Fantacci, J. Godwin, C. Jones, T. Hennigan,

- M. Hessel, S. Kapturowski, T. Keck, I. Kemaev, M. King, L. Martens, H. Merzic, V. Mikulik, T. Norman, J. Quan, G. Papamakarios, R. Ring, F. Ruiz, A. Sanchez, R. Schneider, E. Sezener, S. Spencer, S. Srinivasan, W. Stokowiec, and F. Viola, “The DeepMind JAX Ecosystem,” 2020.
- [21] A. Hill, A. Raffin, M. Ernestus, A. Gleave, A. Kanervisto, R. Traore, P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, and Y. Wu, “Stable baselines.” <https://github.com/hill-a/stable-baselines>, 2018.
- [22] M. Hoffman, B. Shahriari, J. Aslanides, G. Barth-Maron, F. Behbahani, T. Norman, A. Abdolmaleki, A. Cassirer, F. Yang, K. Baumli, S. Henderson, A. Novikov, S. G. Colmenarejo, S. Cabi, C. Gulcehre, T. L. Paine, A. Cowie, Z. Wang, B. Piot, and N. de Freitas, “Acme: A research framework for distributed reinforcement learning,” *arXiv preprint arXiv:2006.00979*, 2020.
- [23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017.
- [24] M. Omar, “Github repository, tensornetworks [source code].” <https://github.com/Mo379/TensorNets>, 2022.
- [25] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [26] R. BELLMAN, “A markovian decision process,” *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957.
- [27] A. Buluc, S. Beamer, K. Madduri, K. Asanovic, and D. Patterson, “Distributed-memory breadth-first search on massive graphs,” 2017.
- [28] N. Putri, Tulus, “Implementation and analysis of depth-first search (dfs) algorithm for finding the longest path,” 2011.
- [29] H. Wang, T. Zariphopoulou, and X. Zhou, “Exploration versus exploitation in reinforcement learning: a stochastic control approach,” 2018.

- [30] C. Hubig, I. P. McCulloch, and U. Schollwöck, “Generic construction of efficient matrix product operators,” *Phys. Rev. B*, vol. 95, p. 035129, Jan 2017.
- [31] I. P. McCulloch, “Infinite size density matrix renormalization group, revisited,” 2008.
- [32] D. Perez-Garcia, F. Verstraete, M. M. Wolf, and J. I. Cirac, “Matrix product state representations,” *Quantum Info. Comput.*, vol. 7, p. 401–430, jul 2007.
- [33] “The tensor network.”
- [34] J. K. Terry, B. Black, and A. Hari, “Supersuit: Simple microwrappers for reinforcement learning environments,” 2020.
- [35] K. K. S. D. e. a. Mnih, V., “Human-level control through deep reinforcement learning,” 2015.
- [36] K. K. S. D. e. a. Mnih, V., “Human-level control through deep reinforcement learning.” <https://doi.org/10.1038/nature14236>, 2015.
- [37] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, “Trust region policy optimization,” *CoRR*, vol. abs/1502.05477, 2015.
- [38] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [39] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, Curran Associates, Inc., 2019.
- [40] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga,

- S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. Software available from tensorflow.org.
- [41] E. M. King, Z. Wang, D. A. Weitz, F. Spaepen, and M. P. Brenner, “Correlation tracking: Using simulations to interpolate highly correlated particle tracks,” *Phys. Rev. E*, vol. 105, p. 044608, Apr 2022.
- [42] S. S. Schoenholz and E. D. Cubuk, “Jax m.d. a framework for differentiable physics,” in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020.
- [43] M. C. Engel, J. A. Smith, and M. P. Brenner, “Optimal control of nonequilibrium systems through automatic differentiation,” 2022.